

IF616BIN · UI/UX

Sabores del Cusco

Explora por antojo

Guía gastronómica estática de platos y experiencias culinarias cusqueñas

Platos

Bebidas

Postres

Sopas

Mercados

Picanterías

Festividades

Experiencias

Desarrollo de Software II — IF616BIN

Platos destacados
Colque Galindo, Jean Franco

6 platos emblemáticos para probar
Fecha: 7 de octubre de 2026

github.com/JnFranco/SaboresCusco

DESTACADO SEMANAL

Pachamanca a la tierra cusqueña

Cocción ancestral bajo piedras calientes con carnes, habas y papas nativas. La experiencia colectiva más emblemática del Valle Sagrado.

4.9 · Valle Sagrado, Cusco

Una interfaz gastronómica estática

Contenido

- Hero promocional
- 8 categorías visuales
- 6 platos destacados
- 3 experiencias gastronómicas
- Footer

Alcance

- Flutter + Dart
- UI declarativa
- StatelessWidget
- Responsive
- Accesibilidad

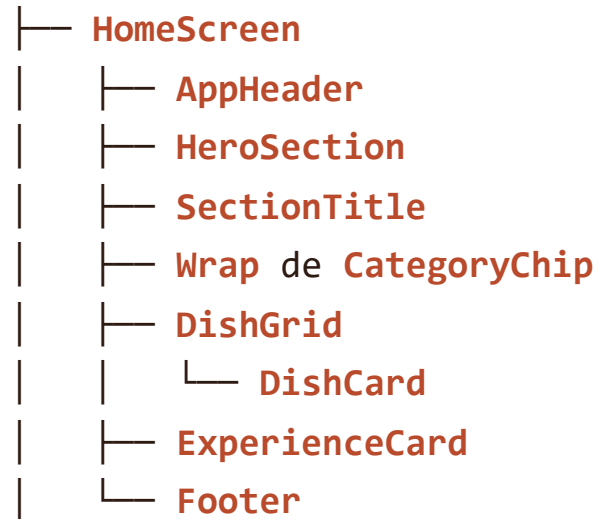
Fuera del alcance

- Backend
- Base de datos
- API
- Autenticación
- Estado
- Navegación

Todo el contenido vive en datos estáticos (`gastronomy_data.dart`) y la pantalla es única: **una sola HomeScreen**.

¿Cómo construimos la interfaz?

MaterialApp



Organización

- `lib/models/` — datos puros (Dish, Experience, Category)
- `lib/data/` — contenido estático
- `lib/screens/` — HomeScreen
- `lib/widgets/` — componentes reutilizables
- `lib/utils/` — constantes y tema

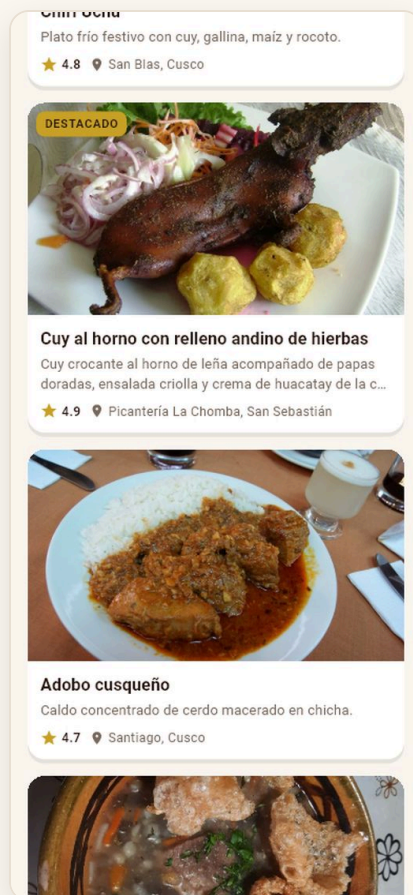
Separación para **reutilización (DRY)** y **pruebas aisladas** de cada componente.

Aplicamos los conceptos trabajados en clase

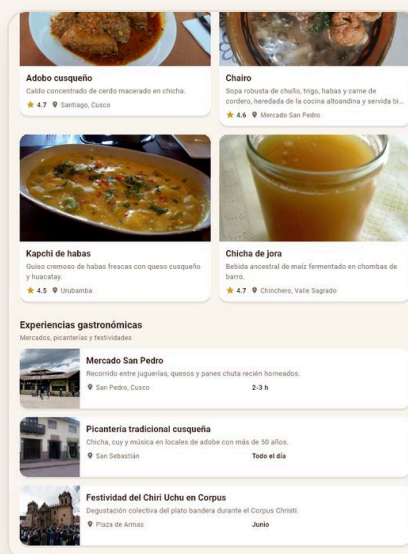
Widget / concepto	Aplicación
Container	Cards, footer y superficies con BoxDecoration
Row / Column	Estructura vertical y horizontal general
Expanded	Reparto del espacio disponible sin tamaños rígidos
Stack + Positioned	Hero (texto sobre imagen) y badge DESTACADO
Wrap	8 chips de categoría de ancho/línea variable
LayoutBuilder	Responsive local (DishGrid, HeroSection)
MediaQuery	Responsive global (padding, header, experiencias)

Cada widget se usa **por necesidad real**: Stack en el Hero para superponer texto; Wrap porque las categorías fluyen a varias líneas.

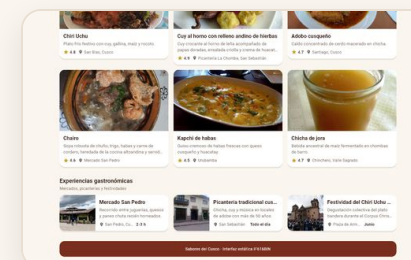
Una interfaz → tres composiciones



390 px
1 columna



768 px
2 columnas



1280 px
3 columnas

< 600 → 1 columna

600 – 899 → 2 columnas

≥ 900 → 3 columnas

El Hero cambia completamente su composición

MOBILE · TABLET (< 900)

Stack + Positioned

DESKTOP (≥ 900)

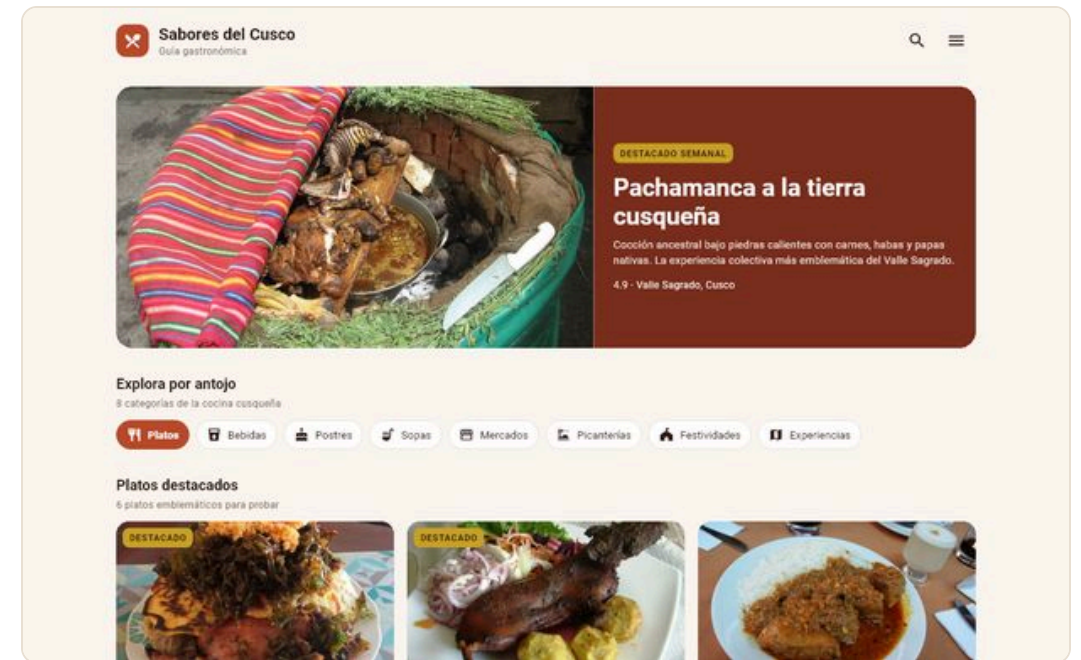
Row + Expanded

Imagen (flex:5) | panel de información (flex:4). **Mejor contraste:** texto sobre panel sólido, sin depender de la fotografía.

¿Cómo evitamos los Overflows?

PROBLEMA

`GridView` dentro de `SingleChildScrollView`

Generaba restricciones de altura no acotadas (**unbounded height**).



SOLUCIÓN

`LayoutBuilder` → `Column` de `Row` → `Expanded(DishCard)`

El `LayoutBuilder` calcula las columnas según el ancho disponible; cada tarjeta recibe un **ancho acotado** = $(\text{ancho} - \text{gaps}) / \text{columnas}$.

También usamos `Expanded`, `Flexible`, `maxLines` con `TextOverflow.ellipsis` y `ConstrainedBox` para prevenir otros overflows.

Primero acotar el espacio, después repartirlo. — La lección central del modelo de restricciones de Flutter. Grid verificado en 8 anchos (273→1024) **sin excepciones**.

No solo apariencia

Contraste WCAG (AA ≥ 4.5)

Tinta / fondo	15.4:1
Secundario / fondo	4.8:1
Blanco / terracota	5.2:1
Tinta / dorado	6.9:1

También verificamos

- ✓ Touch targets de 48 × 48 dp
- ✓ `semanticLabel` en imágenes
- ✓ `Semantics.header` en títulos
- ✓ `ExcludeSemantics` en iconos
- ✓ El color no es el único indicador

QA: blanco/dorado = **2.4:1** (reprobado) → se corrigió a tinta/dorado = **6.9:1**.

Validación automatizada

46 / 46

Tests aprobados

0

issues · flutter analyze

8

anchos de grid · 273 → 1024 px

3

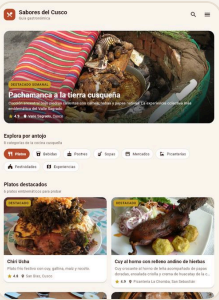
anchos con capturas reales

Robustez

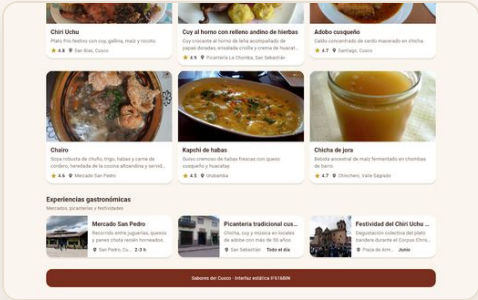
Overflow · contenido extremo · landscape · contraste WCAG · touch targets · semántica



390 px



768 px



1280 px

Qué logramos y qué aprendimos

Resultados

- UI estática desarrollada con Flutter + Dart
- Diseño responsive en 1, 2 y 3 columnas
- Hero adaptativo según el espacio disponible
- Accesibilidad y contraste desde el diseño
- Validación con pruebas automatizadas y visuales
- Sin overflow en los anchos evaluados

Conclusiones

- El modelo de restricciones de Flutter permitió una interfaz adaptable sin errores de layout
- Breakpoints definidos por legibilidad y distribución, no por tamaños de dispositivo
- La accesibilidad se considera durante el desarrollo, no como corrección final
- Responsive no es reducir tamaños: es cambiar la composición cuando el espacio lo requiere
- La separación en componentes mantuvo el código organizado y facilitó las pruebas

**El objetivo no fue solo construir una interfaz que se vea bien,
sino una interfaz que mantenga su estructura, legibilidad y accesibilidad al cambiar
de tamaño**